

Solving Linear Systems

August 2026

1 Introduction

Many problems in numerical analysis, engineering, and scientific computing can be expressed as a system of linear equations:

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n.\end{aligned}$$

This system can be written compactly in matrix form as

$$A\mathbf{x} = \mathbf{b},$$

where

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

Here,

- A is the **coefficient matrix**,
- $\mathbf{x}$ is the **unknown vector**,
- $\mathbf{b}$ is the **right-hand-side vector**.

The objective is to determine $\mathbf{x}$ such that $A\mathbf{x} = \mathbf{b}$.

1.1 Direct and Iterative Methods

Direct methods obtain the solution through a finite sequence of arithmetic operations. In exact arithmetic, they produce the exact solution after a finite number of steps. Methods include Gaussian Elimination and LU Decomposition.

Iterative methods begin with an initial approximation $\mathbf{x}^{(0)}$ and repeatedly improve it

$$\mathbf{x}^{(0)} \rightarrow \mathbf{x}^{(1)} \rightarrow \mathbf{x}^{(2)} \rightarrow \cdots$$

until the approximation satisfies a chosen convergence criterion. Methods include Jacobi Method, Gauss-Seidel Method and Conjugate Gradient Method.

2 Gaussian Elimination

Gaussian Elimination is a **direct method** for solving a system of linear equations $A\mathbf{x} = \mathbf{b}$. The main idea is to transform the original system into an **upper triangular system**, which can then be solved using **back substitution**.

2.1 Forward Elimination

Consider the system

$$\begin{aligned}2x + y - z &= 8, \\ -3x - y + 2z &= -11, \\ -2x + y + 2z &= -3.\end{aligned}$$

Its augmented matrix is

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right].$$

The goal of forward elimination is to eliminate the elements below the diagonal, transforming the matrix into the form

$$\left[\begin{array}{ccc|c} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{array} \right].$$

Pivot :

The **first pivot** is $a_{11} = 2$. To eliminate $a_{21} = -3$, calculate the multiplier

$$m_{21} = \frac{a_{21}}{a_{11}} = \frac{-3}{2}.$$

Then perform the row operation

$$R_2 - m_{21}R_1 \rightarrow R_2.$$

Similarly, to eliminate $a_{31} = -2$,

$$m_{31} = \frac{a_{31}}{a_{11}} = \frac{-2}{2} = -1,$$

and

$$R_3 - m_{31}R_1 \rightarrow R_3.$$

This produces

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 2 & 1 & 5 \end{array} \right].$$

The **second pivot** is $a_{22} = \frac{1}{2}$. To eliminate $a_{32} = 2$, calculate

$$m_{32} = \frac{a_{32}}{a_{22}} = \frac{2}{1/2} = 4.$$

Then perform

$$R_3 - m_{32}R_2 \rightarrow R_3.$$

We obtain

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 0 & -1 & 1 \end{array} \right].$$

The coefficient matrix is now in **upper triangular form**.

2.2 Back Substitution

The resulting system is

$$\begin{aligned}2x + y - z &= 8, \\ \frac{1}{2}y + \frac{1}{2}z &= 1, \\ -z &= 1.\end{aligned}$$

Starting from the last equation,

$$z = -1.$$

Substitute $z = -1$ into the second equation:

$$\frac{1}{2}y + \frac{1}{2}(-1) = 1,$$

which gives

$$y = 3.$$

Finally, substitute $y = 3$ and $z = -1$ into the first equation:

$$2x + 3 - (-1) = 8,$$

so

$$x = 2.$$

Therefore,

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix}.$$

2.3 Partial Pivoting

During forward elimination, the multiplier is calculated as

$$m_{ik} = \frac{a_{ik}}{a_{kk}},$$

where a_{kk} is the current **pivot**. If the pivot $a_{kk} = 0$, the multiplier cannot be calculated. Furthermore, if a_{kk} is very close to zero, division by a small number may amplify round-off errors and lead to numerical instability.

To avoid this problem, Gaussian Elimination is commonly implemented with **partial pivoting**. For each pivot column k , partial pivoting selects the element with the largest absolute value among the entries at or below the current pivot position:

$$|a_{pk}| = \max_{i=k, \dots, n} |a_{ik}|.$$

If $p \neq k$, rows k and p are swapped:

$$R_k \leftrightarrow R_p.$$

For example, consider

$$\left[\begin{array}{cc|c} 0.001 & 1 & 1 \\ 5 & 2 & 7 \\ -2 & 3 & 1 \end{array} \right].$$

In the first column,

$$|5| > |-2| > |0.001|,$$

so the second row is selected as the pivot row. After interchanging R_1 and R_2 ,

$$\left[\begin{array}{cc|c} 5 & 2 & 7 \\ 0.001 & 1 & 1 \\ -2 & 3 & 1 \end{array} \right].$$

The elimination can then proceed using 5 as the first pivot. Partial pivoting improves the numerical stability of Gaussian Elimination and avoids division by zero when a suitable nonzero pivot exists.

2.4 General Algorithm

For a general system

$$A\mathbf{x} = \mathbf{b},$$

Gaussian Elimination with partial pivoting consists of **forward elimination** followed by **back substitution**.

Forward Elimination

For each pivot column $k = 1, 2, \dots, n - 1$, perform the following steps.

1. Select the pivot row

Find the row p containing the largest absolute value in the current pivot column:

$$p = \operatorname{argmax}_{i=k, \dots, n} |a_{ik}|.$$

2. Check the pivot

If $|a_{pk}| < \varepsilon$, the matrix is singular or nearly singular, and the elimination cannot safely continue.

3. Swap rows

If $p \neq k$, interchange rows k and p : $R_k \leftrightarrow R_p$. The corresponding elements of the right-hand-side vector $\mathbf{b}$ must also be interchanged.

4. Eliminate the entries below the pivot

For $i = k + 1, \dots, n$, calculate the multiplier $m_{ik} = \frac{a_{ik}}{a_{kk}}$.

Then update the elements in row i :

$$a_{ij} - m_{ik}a_{kj} \rightarrow a_{ij}, \quad j = k, k + 1, \dots, n,$$

and update the corresponding element of the right-hand-side vector:

$$b_i - m_{ik}b_k \rightarrow b_i.$$

After forward elimination, the coefficient matrix is transformed into an upper triangular matrix.

*** Before performing back substitution, the final pivot must also be checked. If $|a_{nn}| < \varepsilon$, the matrix is singular or nearly singular.

Back Substitution

Starting from the last equation,

$$x_n = \frac{b_n}{a_{nn}}.$$

The remaining unknowns are calculated from bottom to top:

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}, \quad i = n-1, n-2, \dots, 1.$$

3 LU Decomposition

Using Gaussian elimination, we eliminate the entries below the diagonal.

$$A = \begin{bmatrix} 3 & -1 & 2 \\ 6 & -1 & 5 \\ -9 & 7 & 3 \end{bmatrix}$$

Let $l_{21} = \frac{6}{3} = 2$, then we obtain

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ -9 & 7 & 3 \end{bmatrix}.$$

Next, let $l_{31} = \frac{-9}{3} = -3$, which gives

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 4 & 9 \end{bmatrix}.$$

Finally, $l_{32} = \frac{4}{1} = 4$, so that

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 5 \end{bmatrix}.$$

The elimination multipliers are stored in the **lower triangular matrix**:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -3 & 4 & 1 \end{bmatrix}.$$

Therefore,

$$\boxed{A = LU}$$

where

$$\boxed{\begin{bmatrix} 3 & -1 & 2 \\ 6 & -1 & 5 \\ -9 & 7 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -3 & 4 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 5 \end{bmatrix}}$$

The entries of L are the elimination multipliers used during Gaussian elimination. The diagonal entries of L are all equal to 1 because each row retains its original leading coefficient during the elimination process.

The main advantage of LU decomposition is that it allows us to solve a linear system efficiently **when the same coefficient matrix is used with multiple right-hand sides**. Given $Ax = b$, and the decomposition $A = LU$, we obtain $LUx = b$ and $Ux = y$.

Then the system can be solved in two triangular systems: $Ly = b$, followed by $Ux = y$. The first system is solved by **forward substitution**, while the second is solved by **back substitution**.

3.1 LU Decomposition with Partial Pivoting

The previous example assumes that no row interchanges are required. However, in practical numerical computation, partial pivoting may be necessary for the same reasons discussed in Gaussian elimination. Consider

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix}.$$

The first pivot is $a_{11} = 0$. Since the pivot is zero, we cannot perform the elimination directly. Using partial pivoting, we select the largest absolute value in the first column:

$$|2| > |1| > |0|.$$

Therefore, the third row is selected as the pivot row. We interchange R_1 and R_3 :

$$R_1 \leftrightarrow R_3,$$

giving

$$\begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 2 \\ 0 & 2 & 1 \end{bmatrix}.$$

The first pivot is now $a_{11} = 2$. The elimination multipliers are

$$l_{21} = \frac{1}{2}, \quad l_{31} = \frac{0}{2} = 0.$$

Thus,

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 2 & 1 \end{bmatrix}.$$

The second pivot is $a_{22} = \frac{1}{2}$. Therefore,

$$l_{32} = \frac{2}{1/2} = 4.$$

Performing the elimination, we obtain

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 0 & -5 \end{bmatrix}.$$

The elimination multipliers are stored in

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & 4 & 1 \end{bmatrix}.$$

Because we interchanged the first and third rows, we also need a permutation matrix

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

The row interchange means that we decomposed PA , rather than A :

$$\boxed{PA = LU}$$

Indeed,

$$\boxed{\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & 4 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 0 & -5 \end{bmatrix}}$$

3.2 Solving Using LU Decomposition

Once the decomposition $PA = LU$ has been computed, solving $A\mathbf{x} = \mathbf{b}$ does not require performing Gaussian elimination again. Multiplying both sides by the permutation matrix gives $PA\mathbf{x} = P\mathbf{b}$. Since $PA = LU$, we obtain $LU\mathbf{x} = P\mathbf{b}$.

Rather than solving this system directly, we introduce an intermediate vector $\mathbf{y}$ and solve two triangular systems: $L\mathbf{y} = P\mathbf{b}$ followed by $U\mathbf{x} = \mathbf{y}$. The first system is solved by forward substitution, while the second is solved by back substitution.

Forward substitution :

Because L is lower triangular with ones on the diagonal, the unknowns are computed from top to bottom. For the system $L\mathbf{y} = P\mathbf{b}$, the equations are

$$\begin{aligned} y_1 &= (P\mathbf{b})_1, \\ y_2 &= (P\mathbf{b})_2 - l_{21}y_1, \\ y_3 &= (P\mathbf{b})_3 - l_{31}y_1 - l_{32}y_2, \\ &\vdots \\ y_n &= (P\mathbf{b})_n - \sum_{i=1}^{n-1} l_{ni}y_i. \end{aligned}$$

In general,

$$y_i = (P\mathbf{b})_i - \sum_{j=1}^{i-1} l_{ij}y_j, \quad i = 1, 2, \dots, n.$$

Since the diagonal entries of L are all equal to 1, no division is required.

Back substitution :

After obtaining $\mathbf{y}$, we solve $U\mathbf{x} = \mathbf{y}$. Because U is upper triangular, the unknowns are computed from bottom to top. Starting from the last equation,

$$x_n = \frac{y_n}{u_{nn}}.$$

The remaining unknowns are calculated from bottom to top:

$$x_i = \frac{y_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}, \quad i = n-1, n-2, \dots, 1.$$

Unlike forward substitution, division by the diagonal entries of U is necessary because they are generally not equal to one.

Regardless of coding, the lower-left entries store the multipliers of L , while the diagonal and upper-right entries store U . The diagonal of L is implicitly equal to 1. Thus, the matrix could be like

$$\begin{bmatrix} U & U & U \\ L & U & U \\ L & L & L \end{bmatrix}$$

So instead of allocating another $n \times n$ matrix, we overwrite those unused positions. This saves roughly half the memory while still allowing us to reconstruct both matrices whenever we need them.